

EdgeASIC v4.8

RTL Implementation and Verification Specification

Primary-Target Host-Assisted INT8 Conv/GEMM and YOLOv8n Compute-Layer Accelerator

Field	Definition
Project identity	Host-assisted INT8 CNN compute accelerator with YOLOv8n compute-layer primary targets
Document purpose	Defines RTL module contracts, implementation order, verification architecture, test matrix, assertions, evidence package, and integration gates.
Primary hardware targets	Conv/GEMM, Conv2D windowing, 1x1/3x3 Conv, stride/padding, residual ADD, partial tiles, Tensor Table, descriptor ring, graph scheduler, split, concat, upsample, maxpool/SPPF, LUT-SiLU, detect-head convolution, raw tensor export.
Host software boundary	Image preprocessing, driver orchestration, CPU-only comparison, final YOLO decode, NMS, and box drawing remain host-side.
Version	v4.8
Status	Implementation and verification specification; no RTL source file generated by this document.

Scope rule: This document intentionally treats the previously labelled future blocks as primary targets. The verification matrix therefore includes TC-PRI test groups for each such block. Decode and NMS remain host-side software in this version because they are post-processing functions, not the primary hardware acceleration target.

Table of Contents

1. 1. Scope, claims, and implementation discipline
2. 2. RTL module hierarchy and integration strategy
3. 3. Global packages, parameters, and coding rules
4. 4. CSR block, descriptor path, Tensor Table, and graph scheduler
5. 5. DMA, local SRAM/BRAM, and Bank State Controller
6. 6. Baseline Conv/GEMM datapath implementation contract
7. 7. Primary-target operator engines
8. 8. Host/IP synchronization and software driver contract
9. 9. Reset, error handling, interrupts, and recovery
10. 10. Performance counters and evaluation instrumentation
11. 11. Verification environment architecture
12. 12. Current-use and primary-target test matrix
13. 13. Assertion plan
14. 14. Regression strategy and evidence package
15. 15. Bring-up sequence and integration gates
16. 16. Risk register and fallback rules
17. Appendix A. Module checklist
18. Appendix B. Descriptor and CSR summary
19. Appendix C. Verification evidence checklist

1. Scope, Claims, and Implementation Discipline

1.1 Project definition

EdgeASIC v4.8 is defined as a host-assisted INT8 inference accelerator. The hardware IP receives prepared INT8 tensor tiles, weights, bias and quantization parameters, executes the supported compute operation, and returns output tensor tiles or raw detection tensors to the host. The host remains responsible for image preprocessing, tensor tiling, descriptor submission, unsupported software functions, final YOLO decode, NMS, and box drawing.

1.2 Primary target classification

Class	Meaning	Included items
Baseline current RTL	Must pass before any primary target is claimed	CSR, descriptor manager, dispatcher, BSC, DMA abstraction, SRAMs, 8x8 systolic array, SDDU, accumulation, requantization, ReLU/passthrough, output packing, performance counters, sticky errors.
Primary target extension	Included in project target and verification matrix	Conv2D window generator, 1x1/3x3 Conv, stride/padding, partial tile masking, residual ADD, Tensor Table, descriptor ring, graph scheduler, split, concat, upsample, maxpool/SPPF, LUT-SiLU, detect-head convolution, raw tensor export.
Host software boundary	Required for full demo but not RTL hardware block	Image preprocessing, INT8 quantization, descriptor generation, CPU-only comparison, final YOLO decode, confidence filtering, NMS, drawing boxes.
Explicit non-claims	Must not be stated unless separately proven	No measured FPS, no Fmax, no FPGA timing closure, no production DDR integration, no hardware decode/NMS, no autonomous full YOLOv8n hardware execution unless every graph block and post-process block is implemented and verified.

1.3 Implementation discipline

- Every RTL module must have a directed unit test before top-level integration.
- Every cross-module valid/ready, bank ownership, descriptor commit, DMA completion, and metadata alignment rule must have an assertion or monitor.
- Primary target features must be tagged with TC-PRI test IDs. A primary target feature without passing TC-PRI tests is documentation-only in the final defense.
- The baseline Conv/GEMM path is the non-negotiable minimum FYP deliverable. Primary target extensions cannot break this baseline.
- Host/IP data layout is part of the RTL contract. Byte ordering, tensor layout, row padding, descriptor fields, and quantization format must be fixed before DMA router RTL is frozen.

1.4 Version-to-version correction

The v4.8 approach replaces the older fragmented documentation set with a consolidated implementation and verification specification. Blocks that were formerly labelled future scope are now included as primary targets and assigned implementation and test responsibilities. The host-assisted model remains the central architecture identity.

2. RTL Module Hierarchy and Integration Strategy

EdgeASIC v4.8 RTL Module Hierarchy - Primary Target Build

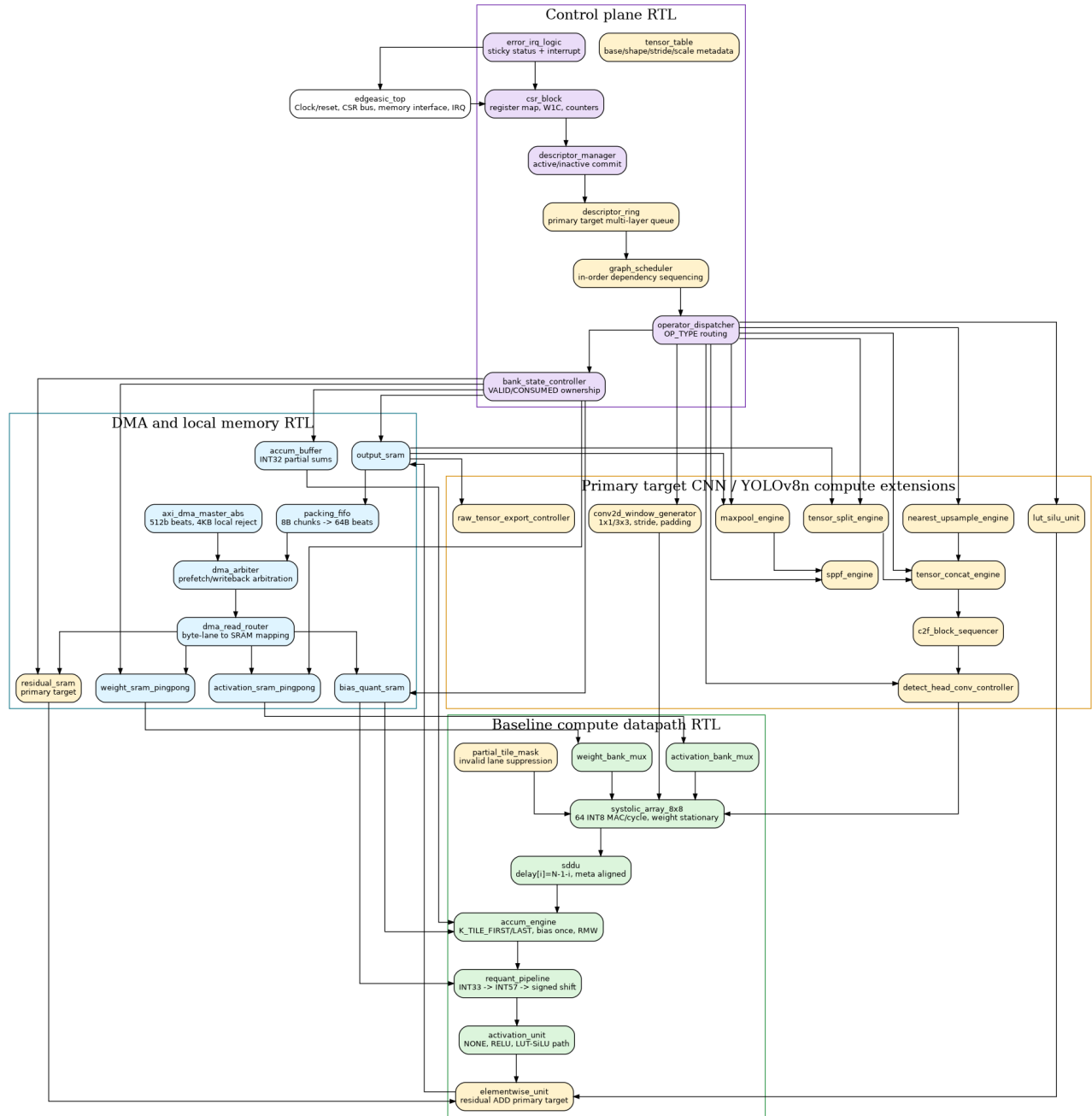


Figure 1. RTL module hierarchy for the primary-target build.

2.1 Top-level integration shell

Item	Requirement
Module	<code>edgeasic_top</code>
Purpose	Integrates CSR/control, memory movement, compute datapath, primary-target engines, error logic, performance counters, and external memory abstraction.
External interfaces	Clock/reset, CSR bus, memory/DMA abstraction interface, interrupt output, optional simulation debug ports.
Internal reset output	Synchronous reset distributed to FSMs, SRAM wrappers, FIFOs, counters, and datapath valid registers.

Item	Requirement
Integration rule	No engine can start directly from host CSRs. All starts pass through descriptor manager, descriptor ring, graph scheduler, and operator dispatcher.

2.2 Ownership split

Owner area	Responsibility
Datapath owner	PE, array, SDDU, accumulation engine, requantization, activation, residual ADD, partial tile mask, Conv2D window generator, LUT-SiLU, maxpool/SPPF datapath, detect-head compute datapath.
Control/memory owner	CSR, descriptor manager, descriptor ring, Tensor Table, graph scheduler, dispatcher, BSC, DMA, SRAM wrappers, packing FIFO, prefetch/compute/writeback FSMs.
Shared verification owner	Both designers write tests for their own blocks. The shared regression harness, scoreboard, and Python golden-model API are jointly reviewed weekly.
Joint integration weeks	Top-level integration, accumulation RMW, residual SRAM/BSC interaction, graph scheduler, and CPU+IP demo are joint tasks.

2.3 Integration order

- Packages/types -> CSR/descriptor -> SRAM/BSC/DMA -> PE array -> SDDU -> accumulation -> requantization -> activation/elementwise -> output packing -> top-level baseline -> performance counters/errors -> primary operator engines -> graph/tensor sequencing -> host-assisted demo.
- Each arrow is a gate. A downstream module may use a stub of an upstream module for early unit testing, but cannot be declared integrated until the real upstream module passes its test ID group.
- The first end-to-end RTL milestone is a single Conv/GEMM tile. Primary-target operators come after the baseline end-to-end test passes.

3. Global Packages, Parameters, and Coding Rules

3.1 Required packages

Package	Required content
config_pkg.sv	Global architectural constants, enum encodings, tile dimensions, SRAM widths, AXI widths, CSR offsets, default reset values.
types_pkg.sv	Signed arithmetic typedefs, packed metadata structs, descriptor structs, Tensor Table structs, status/counter structs.
quant_pkg.sv	Requantization widths, scale/shift rules, saturation functions, activation mode limits.
assert_pkg.sv	Reusable SVA macros or checker modules for valid/ready stability, one-hot states, metadata alignment, and no-deadlock timers.

3.2 Numeric widths

Quantity	Width	Implementation rule
Activation	signed INT8	Two-complement input tensor elements. Symmetric INT8 is the recommended minimum path.
Weight	signed INT8	Two-complement weight values. Per-channel quantization supported through BQ SRAM parameters.

Quantity	Width	Implementation rule
Product	signed INT16	INT8 x INT8 product.
Accumulator	signed INT32	MAC accumulation across K dimension.
Bias sum	signed INT33	INT32 accumulator plus INT32 bias requires one extra sign bit.
Scale	signed INT24	Per-output-channel quantization multiplier.
Scaled product	signed INT57	INT33 x INT24.
Residual intermediate	signed INT9	INT8 output plus INT8 residual before saturation.
Output	signed INT8	Saturated to [-128, 127].

3.3 Dynamic pipeline metadata

Required pipe_meta_t fields

```
typedef struct packed {
    logic        valid;
    logic        k_tile_first;
    logic        k_tile_last;
    logic [ACC_ADDR_W-1:0] acc_addr;
    logic [CHAN_W-1:0] out_chan_base;
    logic [LANES-1:0] lane_valid;
    logic [TENSOR_ID_W-1:0] dst_tensor_id;
} pipe_meta_t;
```

Non-negotiable rule: *K_TILE_FIRST* and *K_TILE_LAST* are dynamic metadata generated by the compute/control FSM for each K-subtile. They are not static descriptor flags and must not be written by host software as descriptor bits.

3.4 Coding rules

- All arithmetic on INT8, INT16, INT32, INT33, and INT57 values must be declared signed at the signal level or explicitly cast before arithmetic.
- Requantization right shift must use arithmetic shift: \$signed(scaled_product) >>> shift_amount.
- No inferred latches. All FSM next-state logic must have defaults for every output and next state.
- All valid payloads must remain stable while valid=1 and ready=0.
- Asynchronous resets are avoided unless required by the selected FPGA board. The default RTL coding style is synchronous active-high reset.
- Primary-target engines may be disabled by synthesis parameter, but if disabled their OP_TYPE must produce a sticky unsupported_op_error rather than hanging.
- No block may silently drop a descriptor. It must either accept, stall with backpressure, or raise a defined error.

4. CSR Block, Descriptor Path, Tensor Table, and Graph Scheduler

4.1 CSR register map

Offset	Register	Access	Definition
0x00	CONTROL	RW	START, SOFT_RESET, PERF_CLEAR, IRQ_ENABLE, descriptor_commit
0x04	STATUS	RO/W1C	DONE, BUSY, ERR_UNSUPPORTED_OP, ERR_UNSUPPORTED_ACT, ERR_AXI_BOUNDARY, ERR_SHAPE, ERR_DMA_RESP, ERR_TIMEOUT
0x08	IRQ_STATUS	RO/W1C	irq_done, irq_error, irq_perf
0x0C	OP_CONFIG	RW	OP_TYPE, ACT_MODE, EW_MODE, engine enable bits

Offset	Register	Access	Definition
0x10-0x3C	Descriptor window	RW	A_BASE, W_BASE, O_BASE, BQ_BASE, INPUT1_BASE, M, N, K, strides, flags, tensor IDs
0x40-0x7C	Tensor Table access window	RW	tensor_id, base, shape, stride, scale id, producer, consumer, valid
0x80	PERF_TOTAL_CYCLES	RO	Cycles from START to DONE or ERROR.
0x84	PERF_COMPUTE_CYCLES	RO	Cycles where compute datapath is active.
0x88	PERF_DMA_WAIT_CYCLES	RO	Cycles stalled waiting for DMA or bank valid.
0x8C	PERF_WRITEBACK_CYCLES	RO	Cycles spent packing or writing output.
0x90	PERF_STALL_CYCLES	RO	Pipeline hold or backpressure cycles.
0x94	PERF_MAC_ACTIVE_CYCLES	RO	Cycles where PE array performs valid MACs.
0x98	PERF_LAYER_COUNT	RO	Completed descriptors/layers.
0x9C	PERF_ERROR_COUNT	RO	Sticky error event count.

4.2 Descriptor fields

Descriptor field group	Implementation requirement
op_type	Selects Conv/GEMM, Conv2D, split, concat, upsample, maxpool, SPPF, LUT-SiLU, C2f sequence, detect-head conv, raw tensor export.
act_mode	ACT_NONE, ACT_RELU, ACT_LUT_SILU. Unsupported modes latch ERR_UNSUPPORTED_ACT.
ew_mode	EW_NONE or EW_ADD for residual addition.
base addresses	A_BASE, W_BASE, O_BASE, BQ_BASE, INPUT1_BASE; can be direct or resolved through Tensor Table.
shape fields	M, N, K plus H, W, C where Conv2D/tensor ops require spatial interpretation.
stride/kernel/padding	kernel_h, kernel_w, stride_h, stride_w, pad_top, pad_bottom, pad_left, pad_right.
flags	ROW_PAD_EN, RESIDUAL_EN, PARTIAL_TILE_EN, PER_CH_QUANT_EN, TENSOR_TABLE_EN, GRAPH_CONTINUE_EN.
tensor IDs	src0_tensor_id, src1_tensor_id, weight_tensor_id, dst_tensor_id, quant_tensor_id.
static only rule	Descriptor flags are static layer-level options. K_TILE_FIRST/LAST are generated dynamically and travel as pipe_meta_t fields.

4.3 Descriptor commit behavior

20. Host writes inactive descriptor window while engine may be idle or busy.
21. Host asserts descriptor_commit. Descriptor manager checks legality, shape, alignment, supported OP_TYPE, and available inactive slot.
22. If accepted, commit_accept=1 for one cycle and the inactive descriptor becomes eligible for graph scheduler or direct dispatch.
23. If busy and no inactive slot exists, commit_accept=0. The current active descriptor must not change.
24. If descriptor is illegal, commit_accept=0 and ERR_SHAPE or relevant error bit latches.
25. Software recovery after error: read STATUS, optionally read debug registers, clear W1C error bits, re-write a legal descriptor, then START again. Automatic restart is forbidden.

4.4 Tensor Table

Topic	Requirement
Purpose	Stores tensor metadata so the graph scheduler can chain layers without host rewriting every address.
Minimum entries	At least 16 entries for FYP prototype. Parameterizable upward.
Fields	valid, base_addr, H, W, C, stride_H, stride_W, row_pad_64B, scale_id, zero_point, producer_desc, consumer_count, layout_id.
Baseline direct mode	If TENSOR_TABLE_EN=0, engine uses descriptor base addresses directly.
Graph mode	If enabled, graph scheduler resolves src/dst tensor IDs to actual base/shape/stride fields.
Verification	Tensor Table must have direct-mode ignore test, read/write test, invalid tensor ID test, and two-layer chain test.

4.5 Graph scheduler

Scheduler rule	Requirement
Scheduling model	In-order descriptor ring execution is the primary target. Out-of-order issue is not required.
Dependency model	A descriptor can start when all source tensor entries are valid and required banks are available.
Completion model	On layer completion, dst_tensor_id is marked valid and producer field is updated.
Unsupported dependency	If a descriptor refers to an unsupported op or invalid tensor, scheduler stalls only if data is pending; otherwise it raises sticky error. It must not hang forever.
Host interaction	Host may run graph mode or single-descriptor mode. The demo can use host-managed layer sequencing if graph scheduler is incomplete, but then graph scheduler cannot be claimed as verified RTL.

5. DMA, Local SRAM/BRAM, and Bank State Controller

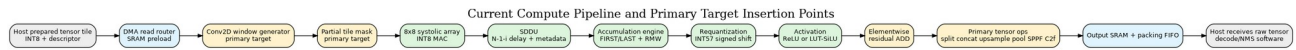


Figure 2. Compute pipeline and primary-target insertion points.

5.1 Memory hierarchy

Memory block	Implementation requirement
External memory abstraction	AXI memory model or board-level DDR bridge. Real production DDR integration is not automatically claimed.
Activation SRAM	Ping-pong INT8 tile banks. Current bank consumed by compute while next bank may be prefetched.
Weight SRAM	Ping-pong INT8 weight banks. Weight-stationary PE loading uses selected bank.
Bias/Quant SRAM	Stores per-channel bias, scale, shift, and optional zero-point metadata.
Accumulation buffer	INT32 partial sums used for K-tile reductions. Must support deterministic read-modify-write.
Residual SRAM	Primary target input bank for EW_ADD residual path.
Output SRAM	INT8 output tile staging before packing FIFO and writeback.
Tensor Table	Metadata store; not tensor payload store.

5.2 AXI DMA boundary and response policy

- DMA requests crossing a 4KB boundary are locally rejected. The DMA abstraction shall not assert external ARVALID/AWVALID/WVALID for the rejected request.
- On rejection, the DMA abstraction asserts dma_done=1 and dma_error=1 and causes STATUS.ERR_AXI_BOUNDARY to latch.
- Requesting FSM treats the request as completed with error and exits its wait state. Deadlock on rejected bursts is forbidden.
- Hardware burst splitting is not required in the primary build; software must pre-align descriptors and tile buffers to avoid 4KB crossings.
- AXI response errors such as SLVERR/DECERR latch ERR_DMA_RESP and complete the request with error.

5.3 DMA read router byte ordering

Mandatory byte-ordering contract

AXI beat byte lane definition:

beat[7:0] -> byte index 0 at lowest memory address

beat[15:8] -> byte index 1

...

beat[511:504] -> byte index 63 at highest address within the 64-byte beat

SRAM lane mapping rule:

SRAM_WORD[element_index] = beat[8*element_index +: 8]

for element_index = 0..63 unless a descriptor-specific tensor layout remap is explicitly selected.

5.4 Bank State Controller states

State	Meaning
INVALID	Bank contents are not usable. Reset state.
FILLING	DMA/router is writing bank. Compute cannot consume.
VALID	Bank contains a complete tile and may be consumed by a matching engine.
CONSUMING	Compute or operator engine owns the bank for read.
CONSUMED	Consumer finished. Bank can be refilled or invalidated.
ERROR_LOCKED	Bank was involved in an error. Requires reset or explicit host recovery.

5.5 BSC invariants

- A bank cannot transition from FILLING to CONSUMING without passing through VALID.
- A bank cannot be written while in CONSUMING.
- VALID/CONSUMED handshake is one ownership transfer per tile.
- Residual SRAM follows the same ownership rules as activation and weight SRAM when EW_ADD is enabled.
- No FSM may wait forever for a bank without a timeout assertion in simulation.

6. Baseline Conv/GEMM Datapath Implementation Contract

6.1 Weight-stationary PE contract

PE topic	Requirement
Dataflow	Weight-stationary. Weights are loaded and held in PE-local registers while activations propagate through the array.
Arithmetic	signed INT8 activation x signed INT8 weight -> signed INT16 product -> signed INT32 partial-sum update.
act_out	Registered one-cycle activation propagation output. Systolic timing assumes this register exists.

PE topic	Requirement
Weight load	Weight load enable is independent from MAC valid. MACs are only valid after weight load completion.
Reset	Clears valid state and local accumulator/control state. Weight data after reset is invalid until explicitly loaded.
Test requirement	Signed positive/negative tests, act_out delay test, weight-hold test, array-level timing test.

6.2 SDDU implementation

The Systolic Drain Deskew Unit aligns diagonal systolic outputs into rectangular channel-major output timing. For N=8, lane 0 is delayed by 7 cycles, lane 1 by 6 cycles, and lane 7 by 0 cycles. The same delay is applied to pipe_meta_t so that data, valid, K-tile metadata, accumulator address, output-channel base, and lane_valid mask remain aligned.

SDDU timing rule

```
for lane i in 0..N-1:
    delay_cycles[i] = N - 1 - i
    data_out[i]    = delay_line_data[i][delay_cycles[i]]
    meta_out[i]    = delay_line_meta[i][delay_cycles[i]]
```

stall rule:

```
if downstream_ready == 0:
    all SDDU delay lines hold both data and metadata together
```

6.3 Accumulation engine

K-tile case	Required behavior
FIRST=1, LAST=1	Single K-tile layer. Bias + current MAC sum -> requantization/output path. No partial buffer dependency.
FIRST=1, LAST=0	First subtile. Bias + MAC sum -> write partial INT32 accumulator. Do not produce final output.
FIRST=0, LAST=0	Middle subtile. Read previous partial, add MAC sum, write updated partial. Do not produce final output.
FIRST=0, LAST=1	Final subtile. Read previous partial, add MAC sum -> requantization/output path. Final partial writeback not required unless debug mode enabled.

Verification warning: The accumulation engine is the most correctness-sensitive datapath block. Bias must be added exactly once per output element. *K_TILE_LAST* controls final drain into requantization; non-last subtiles must not update output SRAM.

6.4 Requantization and activation

Requantization and activation intent

```
// Required arithmetic intent, not full RTL source
biased_sum    = signed_int33(accumulator_int32 + bias_int32)
scaled_product = signed_int57(biased_sum * scale_int24)
shifted       = $signed(scaled_product) >>> shift_amount
saturated_int8 = clamp(shifted, -128, 127)
```

ACT_NONE: output = saturated_int8

ACT_RELU: output = max(saturated_int8, 0)

ACT_LUT_SILU: output = lut_silu(saturated_int8) when enabled and verified

6.5 Partial tile and residual path

Feature	Implementation rule
Partial tile mask	Lane-valid mask suppresses MAC contribution, accumulator writes, output SRAM writes, and performance MAC-active counting for invalid lanes.
Residual ADD	If EW_ADD and FLAG_RESIDUAL_EN are set, residual SRAM must be prefetched before elementwise unit consumption.
Residual arithmetic	INT8 output + INT8 residual -> signed INT9 -> saturate to INT8.
Residual shape rule	Residual tensor shape must match output tile shape. Mismatches raise ERR_SHAPE.
Residual quantization rule	Minimum path assumes host pre-aligns residual quantization. Hardware does not rescale residual unless explicitly added in a later revision.

7. Primary-Target Operator Engines

7.1 OP_TYPE map

OP_TYPE	Class	Meaning
OP_CONV_GEMM	Baseline current	Dense tiled INT8 GEMM / lowered convolution.
OP_CONV2D	Primary target	Conv2D window generation feeding baseline systolic array.
OP_SPLIT	Primary target	Tensor split/view generation.
OP_CONCAT	Primary target	Tensor channel/tile merge.
OP_UPSAMPLE	Primary target	Nearest-neighbor upsample by repeated address pattern.
OP_MAXPOOL	Primary target	Windowed max reduction.
OP_SPPF	Primary target	Repeated maxpool + concat-style tensor path.
OP_LUT_SILU	Primary target	LUT-based SiLU approximation.
OP_C2F_SEQ	Primary target	C2f primitive sequence controller.
OP_DETECT_CONV	Primary target	Detect-head convolution mapping.
OP_RAW_EXPORT	Primary target	Write raw detection tensor format for host decode/NMS.

7.2 Conv2D window generator

- Accepts tensor shape H, W, C and descriptor fields kernel_h, kernel_w, stride_h, stride_w, and padding.
- 1x1 Conv mode may bypass spatial window buffering and feed GEMM directly with channel-major tiles.
- 3x3 Conv mode produces a serialized K dimension equal to kernel_h * kernel_w * input_channels.
- Zero padding generates zero-valued activation lanes without reading out-of-range memory.
- Window generator must preserve the same pipe_meta_t conventions used by the baseline datapath.
- The first implementation target is fixed 1x1 and fixed 3x3 stride-1 padding cases; generalized stride/padding is verified after basic cases pass.

7.3 Tensor split, concat, and upsample engines

Engine	Implementation rule
Split	Creates multiple output tensor views or writes channel ranges to separate destination tensors. Test with exact channel boundary, odd channel boundary, and partial tile boundary.
Concat	Merges input tensors along the channel dimension. Requires shape compatibility except concat axis. Must verify byte layout and row padding.

Engine	Implementation rule
Upsample	Nearest-neighbor repeat in H/W dimensions. Address generator reads each source element multiple times and writes expanded tensor shape.
Routing	These engines may use DMA/memory movement path rather than PE array. They still use descriptor, Tensor Table, BSC, and output SRAM protocols.
Error behavior	Invalid shapes, unsupported axes, and incompatible row padding raise ERR_SHAPE. No silent output corruption allowed.

7.4 Maxpool and SPPF

Block	Rule
Maxpool	Compares INT8 values over a local window. First target is 2x2 or 3x3 window depending on selected model subset. Padding behavior must match Python golden model.
SPPF	Implemented as repeated maxpool and concatenation-style output composition. It may be sequenced by graph scheduler or an SPPF micro-sequencer.
Verification	Maxpool isolated tests must pass before SPPF tests. SPPF tests must compare exact tensor shape and byte layout, not only numerical values.

7.5 LUT-SiLU activation

Topic	Requirement
Input	Signed INT8 activation value after requantization or pre-activation intermediate if selected.
Implementation option A	256-entry INT8 LUT indexed by input value offset by 128.
Implementation option B	Piecewise approximation with small LUT segments. Only allowed if Python golden model uses identical approximation.
Output	Signed INT8 activation result.
Enable rule	ACT_LUT_SILU is primary target. If block is disabled by parameter, selecting it must raise ERR_UNSUPPORTED_ACT.
Test rule	Compare all 256 INT8 input values against Python LUT reference.

7.6 C2f and detect-head path

Feature	Implementation rule
C2f sequencer	Sequences primitive Conv, split, optional residual/add, and concat operations. It is a controller/micro-sequencer, not a new MAC datapath.
Detect-head convolution	Maps detect-head conv layers to Conv/GEMM or Conv2D path and writes raw detection tensor format.
Raw tensor export	Output tensor must be written in a host-readable layout with documented shape, stride, scale, and channel meaning.
Host boundary	YOLO decode, confidence thresholding, and NMS consume raw tensor on host. Hardware does not draw boxes.

8. Host/IP Synchronization and Software Driver Contract

8.1 Host responsibilities

- Image load, resize/letterbox, normalization, and INT8 quantization.

- Tile and weight packing using the documented NHWC-like layout and 64-byte row padding.
- Descriptor generation and CSR writes.
- Polling DONE/ERROR or handling interrupt.
- Output tensor reconstruction and comparison against golden model.
- Decode, confidence filtering, NMS, and final box drawing in the YOLO demo.
- CPU-only baseline timing for demonstration comparison.

8.2 IP responsibilities

- Accept legal descriptors and reject illegal descriptors with sticky CSR errors.
- Move input tiles to local SRAM/BRAM via DMA abstraction.
- Execute supported compute and tensor operations.
- Expose CSR performance counters for kernel-level evaluation.
- Return output tensor tiles or raw tensors in documented byte layout.
- Never hang silently on unsupported OP_TYPE, unsupported activation, 4KB boundary violation, invalid shape, or DMA response error.

8.3 CPU-only versus CPU+IP demo measurement

Measurement	Definition
CPU-only kernel	CPU runs same INT8 Conv/GEMM or layer reference. Used as software baseline.
IP kernel	IP performance counters measure compute cycles, DMA wait, stalls, and writeback cycles.
Full system CPU-only	CPU preprocess, compute, decode/NMS, and display.
Full system CPU+IP	CPU preprocess, transfer/tiling, IP compute, transfer back, CPU decode/NMS/display.
Expected result	Kernel speedup is the primary hardware claim. Full-system speedup depends on transfer bandwidth and should be reported separately.

9. Reset, Error Handling, Interrupts, and Recovery

9.1 Reset behavior

Block	Reset requirement
CSR registers	Reset CONTROL to 0, STATUS to idle/no-error, performance counters to 0 unless debug preserve is enabled.
Descriptor manager	Active/inactive descriptors invalidated.
Descriptor ring	Head/tail reset to 0, entries invalid.
Tensor Table	Entries invalid unless loaded by host after reset.
BSC	All banks INVALID.
FIFOs	Empty.
Datapath	Valid bits cleared, internal hold registers invalid.
FSMs	IDLE state only.

9.2 Sticky error bits

Error bit	Meaning
ERR_UNSUPPORTED_OP	Unsupported or disabled OP_TYPE selected.
ERR_UNSUPPORTED_ACT	Unsupported or disabled activation selected.
ERR_AXI_BOUNDARY	DMA request crosses 4KB boundary and is locally rejected.
ERR_DMA_RESP	AXI/memory response error.
ERR_SHAPE	Illegal dimensions, incompatible tensor shape, invalid stride/padding, residual mismatch, concat mismatch.

Error bit	Meaning
ERR_TIMEOUT	Simulation/watchdog timeout for an operation that failed to complete.
ERR_TENSOR	Invalid Tensor Table entry or dependency not satisfiable.
ERR_INTERNAL	Assertion-exposed internal fatal error for debug builds.

9.3 Recovery sequence

26. Host observes IRQ_ERROR or STATUS error bit.
27. Host reads STATUS and optional debug registers.
28. Host clears W1C error bits.
29. Host may assert SOFT_RESET if bank/FSM recovery is uncertain.
30. Host rewrites legal descriptor/Tensor Table entries.
31. Host starts operation again. Hardware never auto-restarts from an error state.

10. Performance Counters and Evaluation Instrumentation

Counter	Increment predicate
PERF_TOTAL_CYCLES	Increment every cycle from accepted START to DONE or ERROR.
PERF_COMPUTE_CYCLES	Increment when PE array or active operator engine is doing useful work.
PERF_DMA_WAIT_CYCLES	Increment when a compute/control FSM waits for DMA completion or valid bank.
PERF_WRITEBACK_CYCLES	Increment during output packing and writeback.
PERF_STALL_CYCLES	Increment when valid work exists but pipeline is held by backpressure.
PERF_MAC_ACTIVE_CYCLES	Increment when at least one valid MAC lane is active. Invalid partial lanes are excluded from utilization calculations.
PERF_LAYER_COUNT	Increment after each descriptor completes successfully.
PERF_ERROR_COUNT	Increment once per sticky error event.

Evaluation rule: Performance reporting must show kernel-only and full host-assisted measurements separately. A slow prototype transfer path can hide kernel acceleration and must not be used to discredit the compute IP if the counter-based kernel results are strong.

11. Verification Environment Architecture

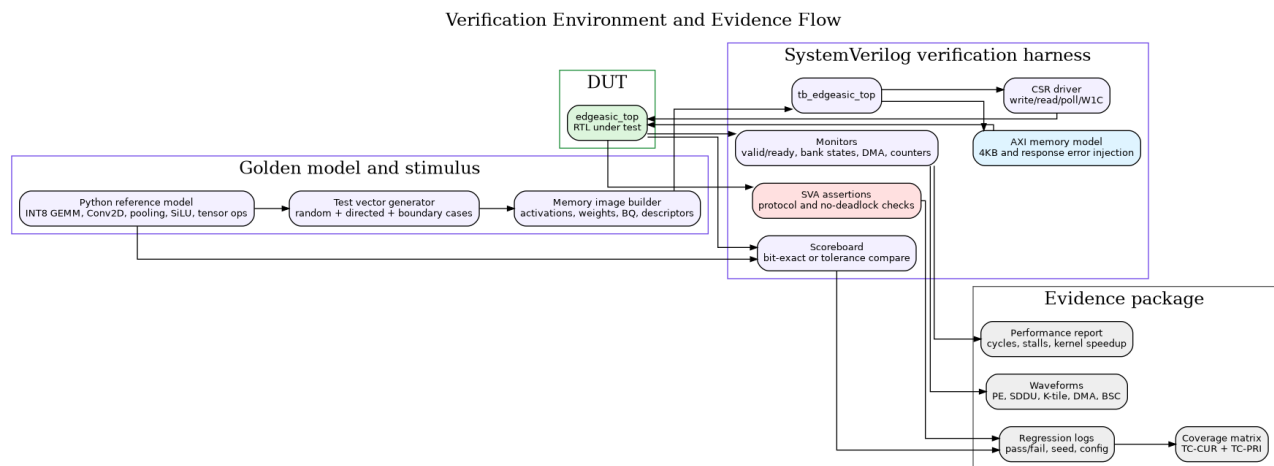


Figure 3. Verification environment and evidence flow.

11.1 Verification layers

Verification layer	Purpose
Unit tests	One testbench per RTL module or tightly coupled block. Required before integration.
Subsystem tests	Control plane, DMA/BSC/memory, Conv/GEMM datapath, primary operator engines.
Integration tests	Full edgeasic_top with memory model and CSR driver.
Graph tests	Descriptor ring + Tensor Table + graph scheduler running two or more operations.
Demo tests	CPU-only versus CPU+IP flow; output comparison plus performance measurement.
Negative tests	Illegal descriptor, unsupported/disabled feature, 4KB crossing, DMA error, invalid tensor ID, shape mismatch.

11.2 Python golden model contract

- Python reference is the source of expected outputs for INT8 GEMM, Conv2D, quantization, ReLU, LUT-SiLU, maxpool, split, concat, upsample, C2f primitive sequence, detect-head mapping, and raw tensor export.
- The golden model must use the same rounding/truncation, saturation, layout, padding, and stride rules as the RTL specification.
- Every test vector stores seed, descriptor, input tensors, weights, BQ parameters, expected output, and tolerance policy.
- For baseline INT8 arithmetic, comparison should be bit-exact. For approximate LUT-SiLU, comparison should be exact against the selected LUT, not against floating-point SiLU unless explicitly specified.

11.3 Scoreboard rules

- Scoreboard compares output SRAM/DMA writeback data against expected byte layout.
- Intermediate probes are used for debug but final pass/fail must be based on externally visible outputs and CSR status.
- For errors, scoreboard verifies that the engine does not start or safely stops, sticky bits latch, and W1C clears them.
- Timeout monitor fails any test that does not reach DONE or ERROR within the expected cycle budget.

12. Current-Use and Primary-Target Test Matrix

12.1 Baseline/current-use test IDs

TC ID	Test name	Blocks	Coverage goal
TC-CUR-001	CSR reset and W1C status	CSR/status	GOOD required
TC-CUR-002	Descriptor commit legal/illegal	descriptor_manager	GOOD required
TC-CUR-003	Unsupported OP_TYPE latches error	dispatcher/CSR	GOOD required
TC-CUR-004	Unsupported/disabled activation error	activation/CSR	GOOD required
TC-CUR-005	DMA 4KB local rejection no deadlock	DMA/FSM/CSR	GOOD required
TC-CUR-006	DMA response error propagation	DMA/FSM/CSR	GOOD required
TC-CUR-007	BSC legal and illegal transitions	BSC	GOOD required
TC-CUR-008	PE signed arithmetic	PE	GOOD required
TC-CUR-009	Registered act_out timing	PE/array	GOOD required
TC-CUR-010	8x8 systolic timing and output order	array	GOOD required
TC-CUR-011	SDDU delay direction	SDDU	GOOD required
TC-CUR-012	SDDU metadata alignment	SDDU	GOOD required
TC-CUR-013	K_TILE_FIRST/LAST generation and four cases	compute/accum	GOOD required
TC-CUR-014	Accumulator RMW hazard	accum_buffer	GOOD required
TC-CUR-015	Requant signed shift negative values	requant	GOOD required
TC-CUR-016	INT8 saturation boundary +127/-128	requant	GOOD required
TC-CUR-017	ReLU and passthrough	activation	GOOD required
TC-CUR-018	DMA read router byte ordering	router	GOOD required
TC-CUR-019	Packing FIFO backpressure	packing_fifo	GOOD required
TC-CUR-020	Performance counters and PERF_CLEAR	CSR/perf	GOOD required

12.2 Primary-target test IDs

TC ID	Test name	Main blocks
TC-PRI-001	Partial tile masking both M and N non-multiple of 8	mask/array/accum
TC-PRI-002	Residual SRAM prefetch and EW_ADD saturation	res_sram/ewu/BSC
TC-PRI-003	Tensor Table direct read/write and baseline ignore mode	tensor_table
TC-PRI-004	Descriptor ring two-entry sequence	descriptor_ring
TC-PRI-005	Graph scheduler two-layer dependency chain	graph_scheduler
TC-PRI-006	1x1 Conv mapped to GEMM	conv2d/win/array
TC-PRI-007	3x3 Conv stride-1 padding case	window_gen/array
TC-PRI-008	General stride/padding boundary tiles	window_gen
TC-PRI-009	LUT-SiLU all 256 INT8 inputs	lut_silu
TC-PRI-010	Maxpool isolated window test	maxpool
TC-PRI-011	SPPF primitive sequence test	sppf/maxpool/concat
TC-PRI-012	Tensor split channel boundary test	split
TC-PRI-013	Tensor concat layout and row padding test	concat
TC-PRI-014	Nearest upsample address repeat test	upsample
TC-PRI-015	C2f primitive sequence test	c2f/graph
TC-PRI-016	Detect-head conv synthetic layer	detect/conv
TC-PRI-017	Raw tensor export layout compatibility	raw_export/host
TC-PRI-018	CPU-only vs CPU+IP kernel comparison	demo/perf
TC-PRI-019	Host-side decode/NMS compatibility with raw tensor	host software
TC-PRI-020	Full primary-target regression smoke suite	top-level

12.3 Negative tests

- Illegal descriptor dimensions M=0, K=0, H/W mismatch, concat mismatch, residual shape mismatch.
- Crossing 4KB DMA request.
- Invalid Tensor Table ID.
- OP_TYPE disabled by build parameter.
- Graph scheduler dependency that cannot be satisfied.
- Stride/padding configuration outside implemented limits.
- Packing FIFO forced full while writeback is active.
- AXI memory model returns error response.

13. Assertion Plan

Assertion ID	Property
A-CSR-001	Sticky error bit remains set until W1C clear.
A-DESC-001	K_TILE_FIRST/LAST are not read from descriptor flags.
A-DESC-002	Active descriptor cannot change while BUSY=1.
A-BSC-001	No bank is written and consumed in same cycle unless explicitly dual-port safe.
A-BSC-002	VALID bank eventually becomes CONSUMED after consumer accepts it.
A-DMA-001	4KB crossing request does not issue external AXI valid.
A-DMA-002	4KB crossing request produces dma_done and dma_error.
A-PE-001	act_out is registered, not combinational pass-through.
A-SDDU-001	Lane delay equals N-1-i.
A-SDDU-002	Data and pipe_meta_t valid are delayed identically.
A-ACC-001	Non-last K tile cannot write final output SRAM.
A-ACC-002	Bias is added only when k_tile_first=1.
A-REQ-001	Shift amount uses arithmetic right shift on signed product.
A-FIFO-001	Packing FIFO data remains stable under backpressure.

Assertion ID	Property
A-GRAPH-001	Graph scheduler starts descriptor only when dependencies are valid.
A-TENSOR-001	Invalid Tensor Table entry prevents engine start and latches error.
A-HOST-001	DONE and ERROR are mutually exclusive for a single operation completion cycle unless configured otherwise.

14. Regression Strategy and Evidence Package

14.1 Regression groups

Regression group	Contents
smoke	Compile, reset, CSR read/write, one tiny GEMM.
current_datapath	PE, array, SDDU, accumulation, requant, activation.
current_control	CSR, descriptor, dispatcher, BSC, DMA, errors.
current_system	End-to-end Conv/GEMM tests including negative values and multi-K tiles.
primary_ops	Conv2D, partial tiles, residual, LUT-SiLU, maxpool, split, concat, upsample.
graph	Tensor Table, descriptor ring, graph scheduler, C2f primitive sequence.
demo	CPU-only vs CPU+IP kernel comparison, raw tensor export, host decode/NMS compatibility.
nightly	All deterministic tests plus randomized seeded tests.

14.2 Required evidence

- Regression summary table with pass/fail, seed, date, and commit ID.
- Waveforms for PE/array timing, SDDU metadata alignment, K_TILE_FIRST/LAST, DMA 4KB error, BSC ownership, graph scheduler layer chain, and raw tensor export.
- Python golden comparison logs with maximum absolute error and exact byte mismatch report if failing.
- Performance counter dumps for kernel-only and host-assisted demo.
- Feature status table: implemented and verified, implemented not fully verified, documentation-only, disabled, or not attempted.

15. Bring-Up Sequence and Integration Gates

Gate	Name	Exit criterion
Gate 0	Spec freeze	Enums, widths, layout, CSR map, descriptor fields frozen.
Gate 1	Control smoke	CSR, descriptor, dispatcher, sticky errors pass.
Gate 2	Memory safety	BSC, SRAM wrappers, DMA 4KB rejection pass.
Gate 3	Compute unit	PE array and SDDU pass.
Gate 4	Accumulation correctness	Four K-tile cases and RMW tests pass.
Gate 5	Quantized output	Requant, saturation, ReLU pass.
Gate 6	First top-level tile	End-to-end Conv/GEMM tile matches Python.
Gate 7	Baseline regression	Current test matrix passes.
Gate 8	Optional/current primary stability	Partial/residual tests pass or are disabled without regression.
Gate 9	Conv2D primary target	1x1 and selected 3x3 tests pass.
Gate 10	Graph/tensor primary target	Tensor Table, descriptor ring, two-layer graph test pass.
Gate 11	YOLO-style primary demo	Selected operator sequence and raw tensor export pass host compatibility test.
Gate 12	Defense package	Documentation, waveforms, logs, demos, safe claims ready.

16. Risk Register and Fallback Rules

ID	Risk	Severity	Mitigation/fallback
R1	Person B overload	High	Both designers write tests; graph/tensor tasks become joint.
R2	SDDU/act_out timing mismatch	High	Waveform review before accumulation integration.
R3	K_TILE_FIRST/LAST bug	Critical	Four-case directed test and assertion before random tests.
R4	Accumulation RMW hazard	Critical	Serialize first implementation if pipelined RMW fails.
R5	DMA byte ordering mismatch	High	Known-pattern 0..63 byte test before top-level.
R6	Graph scheduler complexity	High	Limit primary graph scheduler to in-order two/three-layer chains first.
R7	Conv2D window generator complexity	High	Implement 1x1 first; then fixed 3x3; generalize only after passing.
R8	SPPF/C2f too large	Medium	Implement as primitive sequence and document disabled cases clearly.
R9	CPU+IP full-system speedup hidden by transfer overhead	Medium	Report kernel-only and full-system timing separately.
R10	Overclaim risk	High	Final slides must include explicit non-claim statement.

16.2 Fallback classification rules

- If a primary-target block has RTL but no passing TC-PRI test, it must be marked implemented-but-unverified, not completed.
- If a primary-target block only has architecture and Python model, it must be marked documentation/Python-only.
- If a block is disabled by parameter, selecting it must raise a sticky unsupported error.
- The baseline Conv/GEMM implementation and verification must not be sacrificed to chase primary-target extensions.
- The final defense claim must reflect actual evidence, not plan intent.

Appendix A. Module Checklist

Category	Modules
Top/control	edgeasic_top, csr_block, descriptor_manager, descriptor_ring, tensor_table, graph_scheduler, operator_dispatcher, error_irq_logic, perf_counter_block
Memory	axi_dma_master_abs, dma_arbiter, dma_read_router, activation_sram, weight_sram, bias_quant_sram, residual_sram, accum_buffer, output_sram, packing_fifo, bank_state_controller
Baseline datapath	pe_mac, systolic_array_8x8, sddu, accum_engine, requant_pipeline, activation_unit, elementwise_unit, partial_tile_mask
Primary engines	conv2d_window_generator, tensor_split_engine, tensor_concat_engine, nearest_upsample_engine, maxpool_engine, sppf_engine, lut_silu_unit, c2f_block_sequencer, detect_head_conv_controller, raw_tensor_export_controller
Verification	tb_edgeasic_top, csr_driver, axi_memory_model, scoreboard, python_vector_loader, monitors, assertion bind files, regression scripts

Appendix B. Descriptor and CSR Summary

The descriptor and CSR tables in Chapters 4 and 10 are the authoritative interface summary for v4.8 implementation planning. Any RTL implementation must preserve reset values, access types, W1C semantics, and dynamic metadata separation. If a field is not yet supported by an RTL block, the dispatcher must reject the corresponding OP_TYPE or mode with a defined sticky error rather than ignoring it silently.

Appendix C. Verification Evidence Checklist

- Compiled RTL with no syntax errors.
- All TC-CUR tests passed.
- All claimed TC-PRI tests passed.
- Assertions enabled during regression.
- Golden model version recorded.
- Known failing tests documented.
- Waveforms captured for timing-sensitive blocks.
- Performance counters dumped for demo cases.
- Host raw tensor decode/NMS script tested with exported tensor format.
- Final claim reviewed against evidence table.